# DeepCQF: Making CQF Scheduling More Intelligent and Practicable

Zongrong Cheng, Dong Yang*, Weiting Zhang, Jie Ren, Hongchao Wang, and Hongke Zhang
School of Electronic and Information Engineering, Beijing Jiaotong University, China

ICC 2022 - IEEE International Conference on Communications | 978-1-5386-8347-7/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICC45855.2022.9882280

***Abstract*—Recently, Cyclic Queuing and Forwarding (CQF) based shaper has been proposed by IEEE 802.1Qch to satisfy the demands for deterministic industrial services. However, CQF only defines the queuing model and flow features, the investigations of scheduling for Time-Triggered flows are still at the early stage. How to make the scheduling more efficient and intelligent is still an open issue to be solved. In this paper, we propose a Deep Reinforcement Learning (DRL) assisted two-stage scheduling algorithm, namely DeepCQF, to solve the temporal scheduling problem based on CQF model, which can be adopted in the off-line and on-line scenarios. Due to the high schedulability and low time cost requirements in the off-line and on-line scenarios, respectively, DeepCQF uses the DRL algorithm to take immediate response for the coming flows when scheduling online, while in the off-line scenario, DeepCQF combines the DRL and heuristic algorithms together as a two-stage mechanism to further improve the schedulability. We compare the DeepCQF with other state-of-the-art algorithms for CQF. Experiment results show that DeepCQF can increase the number of accommodating flows by an average of 2 times than Greedy algorithm and the convergence speed by 3 times at most than Double Deep Q-network algorithm.**

***Index Terms*—Time-Sensitive Networking; CQF Model; Deep Reinforcement Learning; Temporal Scheduling.**

## I. Introduction

In recent years, the demands for deterministic network services have been penetrated into a large number of domains, such as industrial automation, automotive control systems, avionic applications, and mobile backhaul, among others [1], [2]. However, the traditional methods in packet switched networks are not suitable for the services of such hard real-time requirements [3]. In order to cope with the problem, the IEEE 802.1 TSN working group has developed a set of promising standards of Time-Sensitive Networking (TSN), which targets at the deterministic forwarding applications and bridging at the link layer of the network, providing these services with deterministic latency, jitter, and high reliability.

TSN provides the per-queue granularity traffic shaping solutions to deal with the synchronized flow forwarding problems that include Time Aware Shaper (TAS, a.k.a. IEEE 802.1Qbv) and Cyclic Queuing and Forwarding (CQF, a.k.a. IEEE 802.1Qch) with minor extensions to standard switches. TAS is required to dynamically plan the configurations of Gate Control List (GCL) to schedule Time-Triggered (TT) flows before the task starts. However, with the flow features changing, the recalculation of GCL configurations can be computational complex. Alternatively, CQF is proposed as an easy-to-use model by installing the static configurations on GCL. By cyclically switching a pair of Ping-Pong queues, CQF guarantees the hop-by-hop deterministic bounded latency. Thus, the latency introduced by flow transmission completely depends on the time interval size (i.e., cycle time [4]) and the number of hops, and is unaffected by any changes of topology or interference from other non-deterministic flows [5].

As IEEE 802.1Qch only provides the queuing model and flow features, the problem intended for the scheduling of TT flows is still an open issue to be solved and optimized. Our original intention of this paper is to explore how to make the temporal scheduling more intelligent and practicable.

Many algorithms have introduced valuable experience for scheduling TT flows in TSN networks. Steiner *et al*. [6] focused on the synthesized schedules based on TAS by using the solver of Satisfiability Modulo Theories (SMT). However, with the increase of the number of flows and nodes in the network, the time cost of SMT will be unacceptable. Yan *et al*. [7] proposed the Tabu-ITP algorithm based on CQF model and got a good trade-off between the mapping quality and computational overhead by regulating the injection time for each flow. Quan *et al*. [5] further proposed a light-weight dynamic algorithm using CQF model to schedule TT flows online, and it greatly improved the number of scheduled flows. Nevertheless, traditional algorithms cannot learn intelligently and actively based on experience, thus it cannot get better over time. Prados-Garzon *et al*. [1] presented a Deep Reinforcement Learning (DRL) solution, LEARNET, to learn scheduling policies based on asynchronous networks, and it achieved higher gains of revenue in 5G backhaul environments. Although DRL has good generalizations for dynamic environments, it is difficult to be as good as the optimization of exact solutions intended for the particular scenario. Therefore, we explore whether we can combine the learning and traditional algorithms to satisfy the different requirements for scheduling TT flows offline and online, and improve the schedulability, resource utilization, and reduce time cost by using CQF model.

In this paper, we propose a DRL assisted two-stage scheduling algorithm, namely DeepCQF, to schedule the TT flows in CQF based TSN networks, which can be adopted in the off-line and on-line scenarios. Specifically, in the on-line scheduling scenario, making quick decisions for the new coming flows is required, thus we design a LearnSched (i.e., Learning based scheduling) algorithm as the first stage of DeepCQF to take the immediate response for each coming flow. In the off-line scenario, we focus on scheduling more TT flows, and a TSSched (i.e., Tabu Search based scheduling) algorithm is

This work was supported in part by the National Key Research and Development Program of China under Grants 2018YFB1702000, and the Fundamental Research Funds for the Central University under Grant 2021YJS006.

The corresponding author is Dong Yang*. E-mail: dyang@bjtu.edu.cn.

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 27,2023 at 07:17:51 UTC from IEEE Xplore. Restrictions apply.

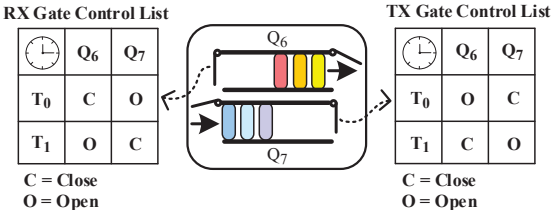
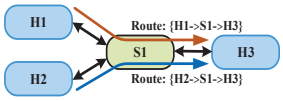



Fig. 1. Cyclic Queuing and Forwarding Model.




(a)

| ID | period | src | dst | latency | jitter | pkt | T | size |
|---|---|---|---|---|---|---|---|---|
| 1.1 | 1600us | H1 | H3 | 1600us | 280us | 1 | 800us | 1KB |
| 1.2 | 1600us | H1 | H3 | 2000us | 300us | 1 | 800us | 1KB |
| 2.1 | 3200us | H2 | H3 | 2000us | 300us | 1 | 800us | 1KB |

(b)

Fig. 2. A simple example of topology and TT flows. (a) An easy-to-understand topology; and (b) Features of the three TT flows with ID 1.1, 1.2, and 2.1, respectively.

further designed as the second stage of DeepCQF to improve the scheduling performance based on the high-quality initial results computed by the LearnSched algorithm.

To evaluate the performance of our proposed algorithm, we compare the DeepCQF with other state-of-the-art algorithms under various system settings. The experiment results show that DeepCQF can schedule nearly twice the flows than Greedy algorithm, and increase the number of schedulable flows by 50%, 40%, and 10% on average compared with SMT, Double Deep Q-Network (DDQN), and Tabu-ITP (Injection Time Planning) algorithms, respectively. And it has the higher temporal resource utilization as well as lower time cost in the off-line and on-line scenarios, respectively. To make the scheduling practicable, we deploy the DeepCQF into our TSN testbed to schedule different TT flows in real environments. In conclusion, the contributions of this paper are:

- We develop a temporal-scheduling model for the scheduling problem of TT flows based on the CQF queuing mechanism and our investigations of the relationship between the schedulability and temporal resource allocation policy.
- We propose DeepCQF, a DRL assisted algorithm firstly used to solve the CQF based scheduling problem of TT flows in TSN networks under the off-line and on-line scenarios.
- We evaluate the performance of the proposed algorithm in simulated environments, and validate the practicability of scheduling by deploying the DeepCQF into our TSN testbed to schedule different TT flows in real environments.

The rest of this paper is organized as follows. In Section II, the scheduling problem is formulated by using the CQF model. Section III proposes the DeepCQF algorithm to solve the formulated problem in the off-line and on-line scheduling scenarios. The performance evaluation, followed by the conclusion are presented in Section IV and V, respectively.

## II. System Model and Problem Formulation

### A. Scheduling Problem based on CQF System Model

The CQF model, which acts as a queuing mechanism for shaping TT flows, providing each flow with predictable and extremely bounded latency by cyclically switching the sending and receiving status of Ping-Pong queues on each device port. The gate operations are executed repeatedly controlled by GCL that takes the hyper period as an operation cycle [8], [9]. As shown in Fig. 1, assuming that the hyper period is divided by two time intervals $T_0$ and $T_1$. In the even time interval $T_0$, $Q_6$ opens the TX gate to send flows to the downstream devices, and $Q_7$ opens the RX gate to receive flows from the upstream devices. In the odd time interval $T_1$, the operations execute conversely. Furthermore, once the packets are transmitted from the current device during a time interval $T_i$, they are required to be received by the next hop within the same duration $T_i$, where $i = \{0, 1\}$. Therefore, the end-to-end latency of each flow is limited by the lower bound $(N-1) \times T$ and upper bound $(N+1) \times T$ with the variation (i.e., jitter) no more than $2 \times T$, where $N$ and $T$ denote the number of hops through the path and the time interval size, respectively [4].

The proposed optimization problem is motivated by scheduling on a simple topology based on the CQF model as shown in Fig. 2 and Fig. 3, where each packet should be allocated to a time interval from the source station to inject for transmission. If all the packets are sent during time interval $T_0$ from the hosts, shown in Fig. 3(a), they will be aggregated to the same buffer (i.e., the queuing space used to store packet data) of switch $S1$. Then in the next time interval, the packet of flow with ID 2.1 which exceeds the maximum capacity of $T_1$ will be dropped or delayed to transmit to host H3 that leads to the unpredictable delay. However, if we delay the TX time of this packet to time interval $T_1$ from the source station in Fig. 3(b), all the packets will be scheduled successfully in the next time interval with bounded end-to-end latency, and the time interval utilization can be improved. Therefore, the schedulability depends on the quality of the temporal resource (i.e., time interval) allocation policy for each flow.

### B. Problem Formulation

We use the directed graph $G = \{V, E\}$ to model the network physical topology, where $V = (H \cup S)$ is a set of nodes, including the end stations $H$ and switches $S$, and $E = \{(i,j) | i, j \in V$ and $i, j$ are connected by a link$\}$ refers to a set of tuples of directed edges linking any two nodes. The size of the time interval is $T$, and the set of TT flows is denoted as $F$. Then each TT flow can be described in Eq. (1).

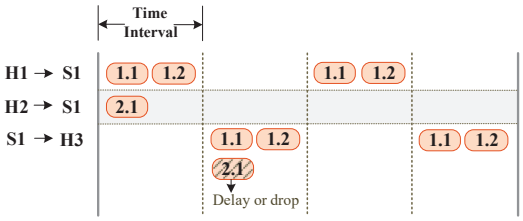
$$f_i = \{id, period, src, dst, path, l_{e2e}, j_{e2e}, pkt_{num}, size, tx\}, \quad \forall f_i \in F, i = 1, 2, \ldots, |F|, \tag{1}$$

where each flow $f_i$ has the features that include ID number, flow period, source address, destination address, pre-defined transmission path, end-to-end latency requirement, jitter requirement, packet number, packet size as the constants we have already known, and TX time as the variable waiting to be solved. The TX time of a flow refers to the timestamp when the flow sends from the Network Interface Card (NIC) of the device. $|F|$ represents the total number of TT flows. We abstract the objective function and core constraints as follows:

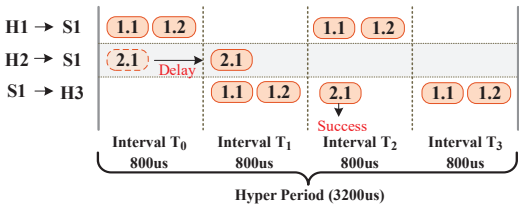
**Objective function:** The optimization goal of the problem is to maximize the number of schedulable flows (i.e., schedulability) by properly allocating the temporal resources.

$$Maximize \quad |F_{suc}|, \quad F_{suc} \subseteq F, \tag{2}$$

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 27,2023 at 07:17:51 UTC from IEEE Xplore. Restrictions apply.




(a) Inappropriate time interval allocation approach.




(b) Appropriate time interval allocation approach.

Fig. 3. Description of the time interval allocation problem.

where $|F_{suc}|$ is the number of successfully scheduled flows.

**Constraints:** We illustrate the important constraints for the network configurations as follows. And upper designers can add other restrictions based on their specific requirements.

• **TX time constraint:** This constraint requires that the TX time of each TT flow should be smaller than its flow period to avoid the packets in different periods preempting the limited resource of NIC at the same time, described by Eq. (3).

$$0 \leq f_i.tx < f_i.period, \quad i = 1, 2, \ldots, |F|. \tag{3}$$

• **Hyper period constraint:** The hyper period denoted as $HP$ is a constant value that should be equal to the least common multiple of the set of flow periods, because the GCL requires periodically executing the operations for all flows.

$$HP = LCM(F.periods). \tag{4}$$

• **Time interval size constraint:** The lower and upper bounds of the time interval size $T$ are given in this constraint. For the minimum value of $T$, the largest packet among TT flows should be accommodated, while for the maximum value of $T$, it is required not to be greater than the greatest common divisor of the set of flow periods. Besides, either for the hyper period or flow period, it should be divisible by $T$.

$$\begin{cases} T \geq \dfrac{max(F.sizes)}{B} + d^{prop} + d^{proc} + d^{queue} + d^{sync}, \\ T \leq GCD(F.periods), \\ f_i.period \,\%\, T = 0, \quad i = 1, 2, \ldots, |F|, \\ HP \,\%\, T = 0, \end{cases} \tag{5}$$

where $B$ denotes the link bandwidth, and $d^{prop}$, $d^{proc}$, $d^{queue}$, $d^{sync}$ represent the packet delays for propagating, processing, queuing, and synchronizing in the worst cases between any two connected devices, respectively.

• **Packet transmission constraint:** The size of total packets transmitted within each time interval should be no more than the maximum time interval capacity, because all the packets stored in the sending buffer are required to be transmitted before the next time interval starts.

$$\begin{cases} 0 \leq \sum\limits_{m=1}^{|F|} \sum\limits_{s=1}^{f_m.pkt_{num}} f_m.size \times \Phi_i(pkt_m^s) \leq C_{max}^i, \\ \lambda = HP/T, \\ i = 1, 2, \ldots, \lambda, \end{cases} \tag{6}$$

where $pkt_m^s$ denotes the $s$th packet of flow $f_m$. And $\Phi_i(\cdot)$ is an indicator function. If the packet $pkt_m^s$ is allocated into time interval $T_i$, then $\Phi_i(pkt_m^s)$ will be 1, otherwise, it will be 0. $\lambda$ is the frequency of appearance of time intervals in a hyper period. $C_{max}^i$ is the maximum capacity of time interval $T_i$.

• **Flow requirements constraint:** We need to ensure that the requirements of end-to-end latency, jitter, and near-zero packet loss for each flow should be satisfied in the worst case.

$$\begin{cases} (N_i + 1) \times T \leq f_i.l_{e2e}, \\ \left| pkt_i^j.l_{e2e} - \dfrac{1}{f_i.pkt_{num}} \sum\limits_{m=1}^{f_i.pkt_{num}} pkt_i^m.l_{e2e} \right| \leq f_i.j_{e2e}, \\ \left| \sum\limits_{m=1}^{f_i.pkt_{num}} \sum\limits_{t=1}^{\lambda} \Phi_t(pkt_i^m) - f_i.pkt_{num} \right| < 1 \times 10^{-6}, \\ i = 1, 2, \ldots, |F|, \end{cases} \tag{7}$$

where $N_i$ represents the number of hops that flow $f_i$ traverses through. Moreover, each packet can be allocated with no more than one time interval, described by Eq. (8).

$$\begin{cases} \sum\limits_{t=1}^{\lambda} \Phi_t(pkt_i^j) \leq 1, \Phi_k(pkt_i^j) \geq 0, \Phi_k(pkt_i^j) \in Z, \\ i = 1, 2, \ldots, |F|, \; j = 1, 2, \ldots, f_i.pkt_{num}, \\ k = 1, 2, \ldots, \lambda. \end{cases} \tag{8}$$

To tackle such an NP-hard temporal scheduling problem [7] in the off-line and on-line scenarios, respectively, we design a DRL assisted two-stage scheduling algorithm, namely DeepCQF, that is introduced in the next section.

## III. DeepCQF: DRL Assisted Two-Stage Scheduling Algorithm

Compared with the non-learning algorithms, DRL can actively learn from the dynamic environment and has lower time cost when testing. Hence, DRL is more preferable to be applied into the on-line scenario. However, in the off-line scenario, the results learned from DRL are not easy to be better than traditional algorithms because the training process for DRL is too generalized. In order to improve the quality of results, the heuristic algorithm is added as the second stage of DeepCQF to schedule the static flows.

**LearnSched design:** As the first stage of DeepCQF, LearnSched uses the DDQN framework for DRL to schedule the incrementally added TT flows with random source and destination addresses online. Then we transform the formulated scheduling problem into a Markov Decision Process (MDP), and the core elements of LearnSched are designed as follows:

**State space:** The state space denoted as $\mathcal{S}$ covers the buffer utilization and flow features at each step. Specifically, for the step $t$, the agent obtains the state $s_t = s_t^{occ} \cup s_t^{flow}$, $s_t \in \mathcal{S}$, where $s_t^{occ}$ comprises the usage of each receiving buffer of switches during each time interval within a hyper period, and $s_t^{flow}$ includes the features of flow $f_t$ coming at step $t$.

$$\begin{aligned} s_t^{flow} = \{ & f_t.id, f_t.period, f_t.src, f_t.dst, f_t.path, \\ & f_t.l_{e2e}, f_t.j_{e2e}, f_t.pkt_{num}, f_t.size\}, \end{aligned} \tag{9}$$

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 27,2023 at 07:17:51 UTC from IEEE Xplore. Restrictions apply.

$$
\begin{aligned}
s_t^{occ} = \{&\xi_{T_1}^{Q_{n_1}^{p_1}}(t), \xi_{T_1}^{Q_{n_1}^{p_2}}(t), \ldots, \xi_{T_1}^{Q_{n_2}^{p_1}}(t), \ldots, \xi_{T_1}^{Q_{n_K}^{p_J}}(t), \\
&\xi_{T_2}^{Q_{n_1}^{p_1}}(t), \xi_{T_2}^{Q_{n_1}^{p_2}}(t), \ldots, \xi_{T_2}^{Q_{n_2}^{p_1}}(t), \ldots, \xi_{T_2}^{Q_{n_K}^{p_J}}(t), \\
&\qquad\vdots \\
&\xi_{T_N}^{Q_{n_1}^{p_1}}(t), \xi_{T_N}^{Q_{n_1}^{p_2}}(t), \ldots, \xi_{T_N}^{Q_{n_2}^{p_1}}(t), \ldots, \xi_{T_N}^{Q_{n_K}^{p_J}}(t)\},
\end{aligned}
$$

where $\xi_{T_i}^{Q_{n_k}^{p_j}}(t)$ represents the usage of buffer on the $j$th interface of the $k$th switch during time interval $T_i$.

**Action space:** The action space $\mathcal{A}$ is a set of time intervals waiting to be allocated for each flow. At each step $t$, the agent takes action $a_t$ for $f_t$ based on the policy $\pi$: $\mathcal{S} \rightarrow \mathcal{A}$ to make the Q-value $Q(s_t, a_{f_t})$ optimal with probability of $\epsilon$.

$$
a_t = \begin{cases} \arg\max\limits_{a_{f_t} \in \mathcal{A}} Q(s_t, a_{f_t}), & \epsilon \geq 0.5 - \beta \times t, \\ a_{f_t}, \quad \epsilon < 0.5 - \beta \times t, \\ \epsilon, \beta \in [0, 1], \end{cases} \tag{10}
$$

where $a_{f_t}$ means the time interval allocated to flow $f_t$ to inject, and it has the coarse-grained constraint extended from Eq. (3).

$$
0 \leq a_{f_t} \leq \frac{f_t.period}{T} - 1, a_{f_t} \in Z. \tag{11}
$$

The Q-value $Q(s_t, a_{f_t})$ is calculated by the action-value function $Q^{\pi}(s, a)$ that reflects the long-term revenue for mapping from $s_t$ to $a_{f_t}$, described by Eq. (12).

$$
Q^{\pi}(s, a) = E\left[r_{t+1} + \gamma Q^{\pi}(s_{t+1}, a_{f_{t+1}}) \middle| s_t = s, a_{f_t} = a, \pi\right], \tag{12}
$$

where $r_{t+1}$ is the immediate reward obtained after the action $a_t$ has been taken under state $s_t$.

**Reward:** Once the action $a_t$ has been taken, the immediate reward $r_{t+1}$ can be obtained from the environment as a feedback to evaluate how well the action performed, shown in Eq. (13). Then the agent tends to learn the better policies to maximize its cumulative rewards.

$$
r_{t+1} = \begin{cases} 1, \quad if\ f_t\ is\ successfully\ scheduled, \\ -1, \quad otherwise. \end{cases} \tag{13}
$$

The state transition and training process are shown in Algorithm 1. From line 1 to 9, the time interval $a_t$ is chosen by the agent and allocated to the flow $f_t$ based on the observation of $s_t$ at each step $t$, then the state transits to $s_{t+1}$ where the usage of buffers and the flow features are changed, denoted as $\xi_{T_i}^{Q_{n_k}^{p_j}}(t+1)$ and $f_{t+1}$, respectively. And the immediate reward $r_{t+1}$ can be obtained. The trajectory $\{s_t, a_t, r_{t+1}, s_{t+1}, done\}$ is stored into the replay buffer. From the start of line 10, the agent begins to train itself by sampling a batch of histories from the replay buffer and update the parameters at each step [10]. The LearnSched has two neural networks, which are evaluation network and target network with the same structure. The evaluation network is trained by minimizing the loss function $L$, where $L = \frac{1}{D_b}\sum_{j=1}^{D_b}\left(y_j - Q\left(s_j, a_j; \omega\right)\right)^2$, to update its parameter $\omega$.

$$
y_j = \begin{cases} r_j, \quad if\ done, \\ r_j + \gamma Q'(s'_j, \arg\max\limits_{a'_j \in A} Q\left(s'_j, a'_j; \omega\right); \omega^-), otherwise, \end{cases} \tag{14}
$$

where $D_b$ denotes the number of histories randomly sampled from the replay buffer. And $y_j$ represents the target value described by Eq. (14). Then the parameter of target network $\omega^-$ is replaced by $\omega$ of evaluation network every $L^-$ steps.

**Algorithm 1:** LearnSched for Dynamic Scheduling.

**Input:** Empty replay buffer $D$, Initial network parameters $\omega$ and $\omega^-$, Target network replacement frequency $L^-$, Replay buffer size $N_r$, Training batch size $D_b$, Action space $A$.
**Output:** Optimal policy $\pi^*$.

1 **for** *episode* $e \in \{1, 2, \ldots, M\}$ **do**
2 Initialize state $s$;
3 **for** *step* $t \in \{0, 1, \ldots, K\}$ **do**
4 Select an action $a_t$ from $Q(s_t, a_{f_t}; \omega)$ based on $\epsilon - greedy$ policy;
5 Inject the flow $f_t$ into allocated time interval;
6 Receive an immediate reward $r_{t+1}$;
7 Transit to the next state $s_{t+1}$;
8 Store tuple $(s_t, a_t, r_{t+1}, s_{t+1}, done)$ into $D$;
9 Replace the oldest tuple if $|D| > N_r$;
10 Sample a minibatch of $D_b$ tuples $(s, a, r, s', done)$ from $D$;
11 Define $a^{max}(s'; \omega) = \arg\max\limits_{a' \in A} Q(s', a'; \omega)$;
12 Determine $y_j = \begin{cases} r, & if\ done, \\ r + \gamma Q'(s', a^{max}(s'; \omega); \omega^-), & otherwise; \end{cases}$
13 Perform a gradient decent step with loss $L = E[(y_j - Q(s, a; \omega))^2]$;
14 Reset $\omega^- = \omega$ every $L^-$ steps;
15 **if** *done* **then**
16 break;
17 **end**
18 **end**
19 **end**

**TSSched design:** In order to further improve the performance of scheduling TT flows offline, we design the TSSched algorithm to search optimal solutions in specific static environments, as the second stage of DeepCQF. The pseudo code of TSSched is shown in Algorithm 2.

The high-quality initial results are obtained by LearnSched. The TT flows are divided into two sets, which are the successfully scheduled set $F_{suc}$ and the set $F_{fail}$ that is failed to be scheduled, respectively. The core design of the searching process is shown from line 7 to 13. In the probability of $\delta$, the random searching strategy is used to explore better solutions. Specifically, in the shifting mode, the randomly selected flows that belong to $F_{suc}$ can be exchanged with each other to change the distribution of time interval usage, while in the exchanging mode, one or more flows are randomly selected from $F_{suc}$, and moved to $F_{fail}$. Another searching strategy refers to the domain expert, which is more inclined to select the flows with larger size, lower deadline, longer path length and period to exchange. By iteratively searching the neighbor candidates, the near-optimal results can be computed.

## IV. Performance Evaluation

In this section, we first evaluate the performance of DeepCQF in the off-line and on-line scenarios on the Python implementations, then we validate the practicability of scheduling by deploying the DeepCQF into our TSN testbed.

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 27,2023 at 07:17:51 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 2:** TSSched for Static Scheduling.

**Input:** Flow set $F$, Buffer resource set $BUF$.
**Output:** Mapping from $f_i \in F_{suc}$ to time interval $T_i$.

$cur_{res} \leftarrow LearnSched_init_result(F, BUF)$;
$best_{res}, cursor \leftarrow cur_{res}, 0$;
**while** *(not stoppingCondition())* **do**
  $cur_{iter} \leftarrow 0$;
  **while** $cur_{iter} < max_{iter}$ **do**
    $candidateLst.push(cur_{res})$;
    **if** $random() \leq \delta$ **then**
      $rand(shifting(cur_{res}, BUF))$;
      $rand(exchanging(cur_{res}, F_{fail}, BUF))$;
    **else**
      $exp(shifting(cur_{res}, BUF))$;
      $exp(exchanging(cur_{res}, F_{fail}, BUF))$;
    **end**
    $cur_{iter} + +$;
  **end**
  $best_{res} \leftarrow update_best_res(candidateLst)$;
  $cur_{res} \leftarrow update_cur_res(candidateLst)$;
  $cursor + +$;
**end**
**return** $best_{res}$;

### A. Performance Evaluation in Simulated Environments

**The parameter settings are listed as follows:**

• **Settings for DeepCQF:** For LearnSched, we set two fully connected hidden layers, each with 1000 neurons. The sizes of replay buffer and sample batch are set to 1e4 and 64, respectively. We use Adam with 0.0001 learning rate and discounting factor of 0.9 to train the evaluation network parameters. For TSSched, the maximum iterations and repetitions are set to 3000 and 200, respectively, which means there are 200 candidate solutions will be generated in each iteration. The tabu length is set to 500.

• **Settings for TSN network:** We set the benchmark topology with 10 hosts and 2 switches shown in Fig. 4(b). The link bandwidth is 1000Mb/s, and the queue length is 1024 to leave enough buffer space. The size of each packet is in the range of 50B∼1KB as recommended by Industrial Internet Consortium [11]. And 700 TT flows are scheduled under various settings of the time interval size and flow periods.

**The experiment results are listed as follows:**

• **Convergence speed comparison:** Here we set the time interval size to 400us. The flows with the set of periods {1600us, 3200us} are scheduled offline. The iterative searching process is shown in Fig. 5(a). Compared with the Tabu-ITP with exchanging mode [7] and DDQN, DeepCQF increases the convergence speed by nearly three times. This is because DeepCQF is initialized with the results of LearnSched that has a better observation of the global network status compared with greedy and random policies adopted in the initialization of Tabu-ITP and DDQN, respectively. Therefore, DeepCQF can be more easy and quick to obtain the optimal solutions.

• **Schedulability comparison:** We compare the DeepCQF with the other four state-of-the-art algorithms adopted in TSN scheduling problems offline. In Fig. 5(b), DeepCQF

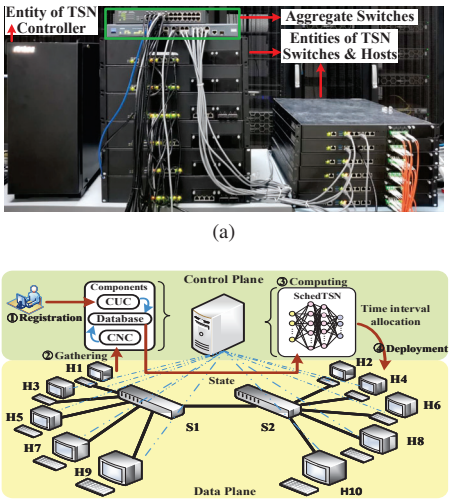



Fig. 4. Architecture of our TSN testbed. (a) Entities of devices; and (b) Depict of the fully centralized architecture.

can schedule more than 600 flows with temporal resource utilization of more than 90% shown in Fig. 5(c). It schedules nearly twice the flows than Greedy, and increases the number of schedulable flows by 50%, 40%, and 10% on average counted in 50 rounds of tests compared with SMT [6], DDQN, and Tabu-ITP algorithms, respectively. Moreover, Fig. 5(b) and Fig. 5(c) show that it has a positive correlation between the schedulability and temporal resource utilization, because the more time intervals are utilized, the more space can be allocated to each flow for transmission.

• **Impact of time interval size:** As shown in Fig. 5(b), with the increase of time interval size, more schedulable flows can be accommodated. One reason is that the capacity of each time interval becomes larger to accommodate more flows, the other reason is that the number of time intervals is reduced within a hyper period that becomes easier to search optimal solutions.

• **Impact of flow periods:** The performances under three different sets of flow periods are shown in Fig. 5(d). For the set of periods {400us, 1600us, 3200us}, the schedulability performs not as good as the other two scenarios, because the flows with the shorter periods need to occupy more temporal resources. In the other two scenarios, they perform similarly, as they suffer more impact from the short period flows.

• **Time cost comparison:** In the on-line scheduling scenario, three groups of flows s1, s2, and s3 with the flow period sets {400us, 1600us, 3200us}, {1600us, 3200us}, and {1600us, 3200us, 4800us} are used for comparing the computational time cost under various algorithms. As shown in Fig. 6(a), group s1 and s2 have nearly the same time cost by scheduling the flows, but for group s3, the time cost is larger. This is because when scheduling the flows of group s1 and s2, the hyper periods are both 3200us, and they have the same number of time intervals to search, while for group s3, the hyper period is 4800us accommodating more time intervals whose searching complexity is increased that leads to higher time cost. Moreover, the time cost for DeepCQF and Greedy are both in the second level. For DeepCQF, the time cost is slightly higher, but varies more gently than Greedy, which can achieve a good trade-off between the value and variation.

Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 27,2023 at 07:17:51 UTC from IEEE Xplore. Restrictions apply.

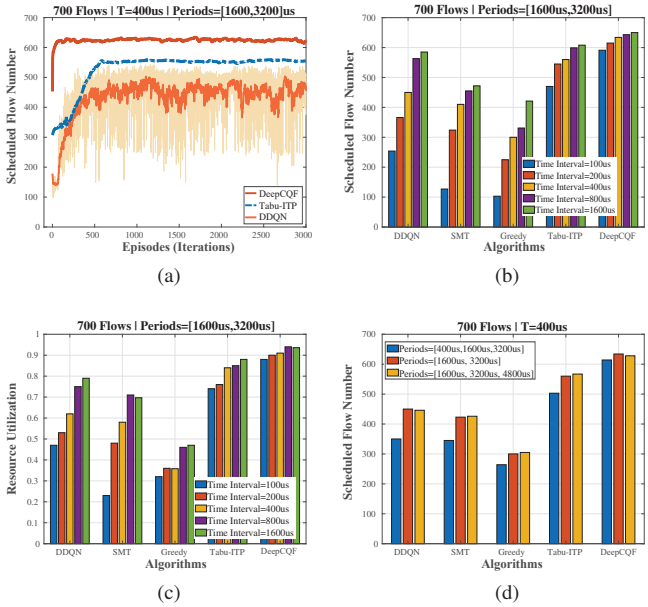



Fig. 5. Performance evaluation. (a) Iterative searching or training process; (b) Impact of time interval size; (c) Temporal resource utilization; and (d) Impact of flow periods.

### B. Scheduling on TSN Testbed in Real Environments

**Implementations on TSN testbed:** As shown in Fig. 4(a), our TSN testbed utilizes the fully centralized architecture according to IEEE 802.1Qcc. We build the benchmark topology, which has 10 hosts and 2 switches, depicted in Fig. 4(b). Each NIC of the devices is INTEL I210. The algorithm is implemented in the Centralized Network Configuration (CNC) of controller with AMD R9 5950X CPU, 32GB of RAM. The CQF based shaper is implemented by using Traffic Control (TC) tool executing the queuing discipline of TAPRIO. We use LinuxPTP to synchronize the system clock. The time interval size is 400us, and the Ring parameters of TX and RX are 1024. The TT flows are created from the SAMPLE-APP-TAPRIO flow generator [12] with the periods {1600us, 3200us}. Taking 50 TT flows with 100,000 packets for scheduling as an example, the experiment procedures and results are listed:

**Step 1: Registration.** The TT flows with the random source and destination addresses are registered by users to Centralized User Configuration (CUC) and stored in the database.

**Step 2: Gathering.** CNC gathers the network status through the Link Layer Discovery Protocol (LLDP).

**Step 3: Computing.** Based on the flow features and the network status extracted from the database, DeepCQF computes the temporal resource allocation policy for each flow.

**Step 4: Deployment.** CNC deploys the GCL and computing results to the hosts and switches through the Network Configuration Protocol (NETCONF) to schedule the flows.

In Fig. 6(b), the scheduling results show that for the packets traversing one switch along the path, the average end-to-end latency is 311.29us with a maximum not exceeding 800us. For the packets transmitting through two switches, the end-to-end latency is on an average of 686.28us and the maximum is not greater than 1200us. The jitter of all the packets is 188.21us on average and is no more than 700.08us. The results

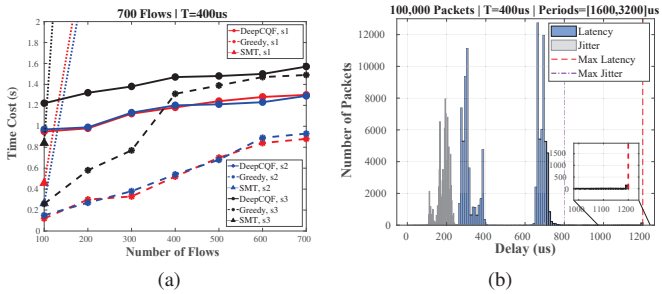



Fig. 6. Experiment results. (a) Time cost comparison; and (b) End-to-end latency and jitter measured on our TSN testbed.

demonstrate that the scheduling on our TSN testbed satisfies the deterministic requirements of end-to-end latency $(N+1)\times T$, which are 800us and 1200us for transmitting through one and two switches, respectively, and the jitter $2 \times T = 800us$ according to the CQF model.

## V. Conclusion

In this paper, we have developed an algorithm named DeepCQF to deal with the temporal scheduling problem in CQF based TSN networks in the off-line and on-line scenarios, respectively. The experiment results have demonstrated that the DeepCQF has higher schedulability, temporal resource utilization, and lower time cost under various system settings. The practicability of scheduling has been validated by running different TT flows on our TSN testbed in real environments. In the future, designing novel mixed flow scheduling algorithms with CQF model would be an interesting direction.

## References


[1] J. Prados-Garzon, T. Taleb, and M. Bagaa, "LEARNET: Reinforcement Learning Based Flow Scheduling for Asynchronous Deterministic Networks," in *Proc. IEEE ICC*, June 2020, pp. 1–6.

[2] A. A. Syed, S. Ayaz, T. Leinmüller, and M. Chandra, "Dynamic Scheduling and Routing for TSN based In-vehicle Networks," in *Proc. IEEE ICC Workshops*, June 2021, pp. 1–6.

[3] A. Nasrallah, V. Balasubramanian, A. Thyagaturu, M. Reisslein, and H. ElBakoury, "TSN Algorithms for Large Scale Networks: A Survey and Conceptual Comparison," *arXiv preprint arXiv:1905.08478*, 2019.

[4] Y. Huang, S. Wang, B. Wu, T. Huang, and Y. Liu, "TACQ: Enabling Zero-jitter for Cyclic-Queuing and Forwarding in Time-Sensitive Networks," in *Proc. IEEE ICC*, June 2021, pp. 1–6.

[5] W. Quan, J. Yan, X. Jiang, and Z. Sun, "On-line Traffic Scheduling Optimization in IEEE 802.1Qch based Time-Sensitive Networks," in *Proc. IEEE HPCC*, Dec. 2020, pp. 369–376.

[6] W. Steiner, S. S. Craciunas, and R. S. Oliver, "Traffic Planning for Time-Sensitive Communication," *IEEE Commun. Standards Mag*, vol. 2, no. 2, pp. 42–47, June 2018.

[7] J. Yan, W. Quan, X. Jiang, and Z. Sun, "Injection Time Planning: Making CQF Practical in Time-Sensitive Networking," in *Proc. IEEE INFOCOM*, July 2020, pp. 616–625.

[8] "IEEE Standard for Local and Metropolitan Area Networks – Bridges and Bridged Networks – Amendment 29: Cyclic Queuing and Forwarding," *IEEE Std. 802.1Qch-2017*, pp. 1–30, June 2017.

[9] "IEEE Standard for Local and Metropolitan Area Networks – Bridges and Bridged Networks - Amendment 25: Enhancements for Scheduled Traffic," *IEEE Std 802.1Qbv-2015*, pp. 1–57, Mar. 2016.

[10] W. Zhang, D. Yang, W. Wu, H. Peng, N. Zhang, H. Zhang, and X. Shen, "Optimizing Federated Learning in Distributed Industrial IoT: A Multi-Agent Approach," *IEEE J. Sel. Areas Commun.*, Oct. 2021.

[11] Industrial Internet Consortium, "Time Sensitive Networks for Flexible Manufacturing Testbed Characterization and Mapping of Converged Traffic Types," Mar. 2019.

[12] Intel Corporation, "TSN Reference Software for Linux," https://github.com/intel/iotg_tsn_ref_sw/tree/apollolake-i/, Apr. 2019.


Authorized licensed use limited to: Beijing Jiaotong University. Downloaded on June 27,2023 at 07:17:51 UTC from IEEE Xplore. Restrictions apply.